

GLP: A Grassroots, Multiagent, Concurrent, Logic Programming Language for AI

Ehud Shapiro

London School of Economics
and Weizmann Institute of Science

A grassroots platform is a multiagent distributed system with multiple independent instances that can form and operate independently of each other and of any global resource, yet may coalesce into ever larger instances, possibly resulting in a single global instance. Grassroots platforms aim to offer an egalitarian/democratic alternative to centralised/autocratic and decentralised/plutocratic global platforms.

Here, we present Grassroots Logic Programs (GLP), a multiagent concurrent logic programming language designed for the implementation of grassroots platforms. To introduce the language, we recall the standard operational semantics of logic programs; introduce the operational semantics of Concurrent (single-agent) GLP as a restriction of that of LP; recall the notion of multiagent atomic transactions; use them to introduce the operational semantics of multiagent GLP; and prove multiagent GLP to be grassroots. The accompanying programming example is the grassroots social graph—the foundational grassroots platform on which all others are based.

The mathematical foundations presented here are being used by AI to derive implementations of GLP as well as to program in GLP, as detailed in companion papers: a workstation-based implementation of Concurrent GLP in Dart was derived from the operational semantics of Concurrent GLP; a smartphone-based implementation of multiagent GLP in Dart/Flutter is being developed based on the operational semantics of multiagent GLP; a moded type system for GLP was designed (and implemented by AI in Dart) to facilitate collaborative human-AI development of GLP programs, where AI derives working GLP programs from human-approved type definitions and declarations; GLP implementations of grassroots platforms for the social graph, social networks, currencies and bonds, and more, are derived by AI from specifications written as volitional multiagent atomic transactions.

While concurrent logic programming and its powerful programming techniques have been known for four decades, adoption has been hampered by their inaccessibility to the average programmer; with AI as a hyper-programmer this limitation is removed. Moreover, our experience suggests that the abstract nature of concurrent logic programming renders it a more powerful and productive language than mainstream languages for collaborative human-AI abstract specifications-based program development.

Keywords: concurrent logic programming, grassroots platforms, operational semantics, multiagent transition systems, AI programming

1 Introduction

Grassroots. Grassroots platforms aim to offer an egalitarian and democratic alternative to centralised and autocratic (Facebook) and decentralised and plutocratic (Bitcoin) global platforms [35]. A *grassroots* platform [32, 38] is a multiagent distributed system with multiple independent instances that can form and operate independently of each other and of any global resource other than the network, yet may coalesce into ever larger instances, possibly resulting in a single global instance. A grassroots platform aims to operate solely on the smartphones of its participants.

Specifications of grassroots platforms that have been provided and proven grassroots include the grassroots social graph [33, 38] — the infrastructure platform on which all others build — grassroots social networks [33, 36], grassroots cryptocurrencies and bonds [34, 18, 37], and grassroots federations [47, 12]; the unifying formal framework being volitional multiagent atomic transactions [20]. Working GLP implementations of the social graph (this paper), child-safe social networks [36], grassroots cryptocurrencies and bonds [37], and secure recovery from major faults [8] have been derived by AI from these specifications. The Scuttlebutt protocol and social network [15] is perhaps the sole example of a deployed grassroots platform to date.

Grassroots Logic Programs. Here, we present Grassroots Logic Programs (GLP), a multiagent, concurrent, logic programming language, designed for the implementation of grassroots platforms. Syntactically, GLP extends Logic Programs (LP) [22] with:

1. **Readers:** Each logic variable X (now referred to as *writer*) is paired with a *reader* $X?$, which is assigned a value only once its paired writer X is assigned that value;
2. **Single-Occurrence (SO):** A variable may occur at most once in a goal or a clause; and
3. **Single-Reader Single-Writer (SRSW):** A writer occurs in a clause iff its paired reader does.

The result eschews unification in favour of simple term matching, and conjures both linear logic [11] and futures/promises [1, 9]: An assignment to a variable may be produced at most once, via the sole occurrence of a writer (promise), and consumed at most once, via the sole occurrence of its paired reader (future).

Hence, in a multiagent distributed implementation, if a writer and its paired reader are held by different agents, an assignment to the writer is realised as a single message from the writer-holding agent to the reader-holding agent. The source of all the powerful concurrent logic programming techniques is that such a message may, in turn, contain further readers and writers. In the simplest case, a reader is used to send subsequent messages (streaming). A writer is used to send a reply to the sender. More generally, variables in a message paired with variables held by third agents can be used for arbitrary network reconfiguration (e.g. friend-mediated introduction in the social graph example below).

Semantics. The operational semantics of GLP is presented in two stages: First, we present *Concurrent GLP* (cGLP), a standard interleaving-based concurrent (single-agent) operational semantics defined as a restriction of standard LP semantics, preserving computation-as-deduction [16].

Second, we present multiagent transition systems and their specification via multiagent atomic transactions, and use them to define the multiagent operational semantics of GLP, *Multiagent GLP* (maGLP), with (1) **Communication:** Named agents that operate independently while communicating to remote readers assignments made to paired local writers; (2) **Cold-calls:** A means for sending a term with variables to a named agent while retaining their paired variables locally. Cold-calls allow two agents in disconnected components of the social graph to become the owners of paired logic variables. We illustrate maGLP via the grassroots social graph, in which agents may initiate friendships (bidirectional communication channels) via “cold-calls” as well as introduce mutual friends to each other. Lastly, we recall the definition of grassroots platforms and how to prove that a platform specified via atomic transactions is grassroots [32, 38], and apply these to prove that maGLP is indeed grassroots.

AI. The mathematical foundations presented here are being used by AI to derive implementations of GLP, as well as to program in GLP, as detailed in companion papers: a workstation-based implementation of cGLP in Dart was derived from the operational semantics of cGLP; a smartphone-based implementation of maGLP in Dart/Flutter is being developed based on the operational semantics of maGLP [39]. A moded type system for GLP was designed (and implemented by AI in Dart) to facilitate collaborative

human-AI development of GLP programs, where AI derives working GLP programs from human-agreed type definitions and declarations [40]. A secure implementation of GLP using public-key cryptography, mutual attestations, and friend-based identity custodians is being developed [8]. Example GLP programs in this paper are typed.

Paper outline. Section 2 presents GLP. Section 3 extends to multiagent GLP: multiagent transition systems, the maGLP definition, and the multiagent social graph. Section 4 proves maGLP is grassroots. Section 5 reviews related work, and Section 6 concludes. All proofs and supporting material appear in the extended (arXiv) version of this paper.

2 GLP

We present GLP syntax, operational semantics, and programming techniques.

2.1 Syntax

GLP extends LP by adding a paired *reader* $X?$ to every “ordinary” logic variable X , now called a *writer*.

Definition 2.1 (GLP Variables). Let $\mathcal{V}$ denote the set of LP variables (identifiers beginning with upper-case), henceforth called *writers*. Define $\mathcal{V}^? = \{X? \mid X \in \mathcal{V}\}$, called *readers*. The set of all GLP variables is $\hat{\mathcal{V}} = \mathcal{V} \cup \mathcal{V}^?$. A writer X and its reader $X?$ form a *variable pair*.

GLP terms, unit goals, goals, and clauses are as in LP but defined over the variables in $\hat{\mathcal{V}}$.

Definition 2.2 (Single-Occurrence (SO) Invariant). A term, goal, or clause satisfies the *single-occurrence (SO) invariant* if every variable occurs in it at most once.

Definition 2.3 (GLP Program, Goals). A clause C satisfies the *single-reader/single-writer (SRSW) restriction* if it satisfies SO and a variable occurs in C iff its paired variable also occurs in C . A *GLP program* is a finite sequence of clauses satisfying SRSW; clauses for the same predicate form a *procedure*. The set of GLP goals $\mathcal{G}(P)$ includes all goals over $\hat{\mathcal{V}}$ and the vocabulary of P that satisfy SO.

The purpose of the SRSW restriction is to prevent multiple writer occurrences racing to assign a variable; the simpler (SW) restriction is insufficient, since if a reader with multiple occurrences is assigned a term with a writer, the result is a writer with multiple occurrences.

Example 2.4 (Fair Merge). Consider the quintessential concurrent logic program for fairly merging two streams, written in GLP:¹

```
Stream ::= [] ; [_ | Stream] .
```

```
procedure merge(Stream?, Stream?, Stream) .
merge([X|Xs], Ys, [X?|Zs?]) :- merge(Ys?, Xs?, Zs) .
merge(Xs, [Y|Ys], [Y?|Zs?]) :- merge(Xs?, Ys?, Zs) .
merge(Xs, [], Xs?) .
merge([], Ys, Ys?) .
```

and the goal `merge([1,2,3|Xs?], [a,b|Ys?], Zs)`. The goal satisfies SO and each clause satisfies SRSW. The first clause swaps inputs in the recursive call, ensuring fairness when both streams are available.

As we shall see (Proposition 2.12), the SO invariant is maintained by the SRSW restriction: reducing a goal satisfying SO with a clause satisfying SRSW results in a goal satisfying SO.

¹Mode annotations ($? \text{ suffix}$) and type declarations (`procedure ..., T ::= ...`) used informally in examples are formally introduced in the Typed GLP companion paper [40].

2.2 Operational Semantics

Definition 2.5 (Transition System). A *transition system* is a tuple $TS = (C, c_0, T)$ where C is an arbitrary set of *configurations*, $c_0 \in C$ is a designated *initial configuration*, and $T \subseteq C \times C$ is a *transition relation*, with transitions written $c \rightarrow c' \in T$. A transition $c \rightarrow c' \in T$ is *enabled* from configuration c . A configuration c is *terminal* if no transitions are enabled from c . A *computation* is a (finite or infinite) sequence of configurations where for each two consecutive configurations (c, c') in the sequence, $c \rightarrow c' \in T$. We write $c \xrightarrow{*} c'$ to denote the existence of a computation (empty if $c = c'$) from c to c' . A *run* is a computation starting from c_0 , which is *complete* if it is infinite or ends in a terminal configuration. The *outcome* of a complete run is determined by a domain-specific function from complete runs to an outcome space.

Definition 2.6 (Substitutions and Assignments). A GLP *writer assignment* is a term of the form $X := T$, $X \in \mathcal{V}$, $T \notin \mathcal{V}$, satisfying SO. Similarly, a GLP *reader assignment* is a term of the form $X? := T$, $X? \in \mathcal{V}?$, $T \notin \mathcal{V}$, satisfying SO. A *writers (readers) substitution* σ is the substitution implied by a set of writer (reader) assignments that jointly satisfy SO. Given a writers assignment $X := T$, its *readers counterpart* is $X? := T$, and given a writers substitution σ , its *readers counterpart* $\sigma?$ is the readers substitution defined by $X?\sigma? = X\sigma$.

Definition 2.7 (GLP Renaming, Renaming Apart). A GLP *renaming* is an injective substitution $\rho : \hat{\mathcal{V}} \rightarrow \hat{\mathcal{V}}$ such that for each $X \in \mathcal{V}$: $X\rho \in \mathcal{V}$ and $X?\rho = (X\rho)?$. Two GLP terms *have a variable in common* if for some writer $X \in \mathcal{V}$, either X or $X?$ occurs in both. A GLP renaming σ *renames T' apart from T* if $T'\sigma$ and T have no variable in common.

Definition 2.8 (Writer MGU). Given two GLP unit goals A and H , a *writer mgu* is a writers substitution σ such that $A\sigma = H\sigma$ and σ is most general among such substitutions.

Definition 2.9 (GLP Goal/Clause Reduction). Given GLP unit goal A and clause C , with $H :- B$ being the result of the GLP renaming of C apart from A , the *GLP reduction* of A with C *succeeds with result* (B, σ) if A and H have a writer mgu.

Definition 2.10 (cGLP Transition System). Given a GLP program P , an *asynchronous resolvent* over P is a pair (G, σ) where $G \in \hat{\mathcal{G}}(P)$ and σ is a readers substitution.

A transition system $cGLP = (\mathcal{C}, c_0, \mathcal{T})$ is a *cGLP transition system* over P and initial goal G_0 satisfying SO if:

1. $\mathcal{C}$ is the set of all asynchronous resolvents over P
2. $c_0 = (G_0, \emptyset)$
3. $\mathcal{T}$ is the set of all transitions $(G, \sigma) \rightarrow (G', \sigma')$ satisfying either:
 - (a) **Reduce:** there exists unit goal $A \in G$ such that $C \in P$ is the first clause for which the GLP reduction of A with C succeeds with result $(B, \hat{\sigma})$, $G' = (G \setminus \{A\} \cup B)\hat{\sigma}$, and $\sigma' = \sigma \circ \hat{\sigma}?$
 - (b) **Communicate:** $\{X? := T\} \in \sigma$, $X? \in G$, $G' = G \setminus \{X?\} \cup T$, and $\sigma' = \sigma$

cGLP Reduce differs from LP in (1) the use of a writer mgu instead of a regular mgu and (2) the choice of the first applicable clause instead of any clause. The first is the fundamental use of GLP readers for communication and synchronization. The second compromises on the or-nondeterminism of LP to allow writing fair concurrent programs, such as fair merge above. Note that or-nondeterminism is not completely eliminated, as different scheduling of arrival of assignments on the two input streams of merge may result in different orders in its output stream.

The cGLP Communicate rule realizes the use of reader/writer pairs for asynchronous communication: it communicates an assignment from its writer to its paired reader.

Persistence of Reducibility. Key differences between LP and cGLP relate to persistence. In LP, if a goal cannot be reduced, it will never be reduced. In cGLP, a goal that cannot be reduced now may be reduced in the future: if A and H have an mgu that writes on a reader $X? \in A$, and therefore have no writer mgu at present, another goal that has X may reduce, assigning X , and later $X?$, to a value that will allow A and H to have a writer mgu. Conversely, in LP, if a goal A can be reduced now with some clause $H : -B$, with a regular mgu of A and H , it may not be reducible in the future due to variables that A shares with other goals being assigned values by reductions of other goals, preventing unification between the instantiated A and H . In cGLP, if a goal A can be reduced now (with a writers mgu), it can always be reduced in the future, as the SO invariant ensures that no other goal can assign any writer in A .

Implementation-wise, if a GLP goal A cannot be reduced now, but there is a readers substitution σ such that $A\sigma$ can be reduced, such readers are identified, the goal A *suspends* on these readers, and is rescheduled for another reduction attempt once any of them is assigned.

Despite these differences, cGLP adopts the same notions of proper run, successful run, and outcome as LP, and has the same notion of logical consequence as LP. Let $/?$ be an operator that replaces every reader by its paired writer.

Proposition 2.11 (cGLP Computation is Deduction). *Let $(G_0 : - G_n)\sigma$ be the outcome of a proper cGLP run $\rho : (G_0, \sigma_0) \rightarrow \dots \rightarrow (G_n, \sigma_n)$ of cGLP. Then $(G_0 : - G_n)\sigma/?$ is a logical consequence of $P/?$.*

We note an additional safety property of cGLP runs.

Proposition 2.12 (SO Preservation). *If the initial goal G_0 satisfies SO, then every goal in the cGLP run satisfies SO.*

Fairness requires identifying “the same transition” across configurations as goals are instantiated by reader substitutions. Following [20], we define goal identity and transition equivalence.

Definition 2.13 (Goal Identity). Each unit goal in a cGLP computation carries a unique *identifier* assigned at spawn time: goals in the initial goal G_0 receive distinct initial identifiers, and goals spawned by a Reduce transition receive fresh identifiers. Communicate transitions preserve identifiers: the goal containing $X?$ retains its identity after instantiation.

Definition 2.14 (Transition Equivalence). Given a set of transitions T , a *transition equivalence* [20] is an equivalence relation $\sim$ on T . We write $[t]$ for the equivalence class of t under $\sim$. An equivalence class $[t]$ is *enabled* in configuration c if some $t' \in [t]$ is enabled in c .

For cGLP, $t_1 \sim t_2$ if:

- both are Reduce transitions that reduce the same goal (by identity) with the same clause, or
- both are Communicate transitions that apply the counterpart of the same writer assignment to the same goal (by identity).

Two equivalent Reduce transitions may operate on different instantiations of the same goal—with different reader substitutions applied—but produce the same writer mgu (since the writer mgu assigns only writers and is unaffected by reader substitutions).

Definition 2.15 (cGLP Fair Run). A complete cGLP run is *fair* if for every equivalence class $[t] \in T/\sim$: whenever $[t]$ becomes enabled, eventually some $t' \in [t]$ is taken.

The SO invariant of GLP allows eschewing unification in favour of *term matching*: if two terms that jointly satisfy SO are unifiable, their mgu maps each variable in one term to a subterm of the other. Term matching thus performs joint term-tree traversal and collects variable assignments along the way; the detailed definition and table appear in the extended (arXiv) version of this paper.

2.3 Guards

GLP clauses may include *guards*—tests that determine clause applicability.

Definition 2.16 (Guarded Clause). A *guarded clause* has the form $H :- G \mid B$, where H is the head, G is a conjunction of guard predicates, and B is the body. The guard separator “ $\mid$ ” distinguishes guards from the body, and is interpreted logically as a conjunction. Guard arguments are readers paired to head writers.

Guards have three-valued semantics. Each guard predicate explicitly defines its *success* condition. A guard *suspends* if it does not succeed but some instance of it under a readers substitution would succeed. A guard *fails* if no such instance exists. A guard conjunction succeeds if all members succeed; it suspends if any member suspends and none fail; it fails if any member fails.

Definition 2.9 of a GLP goal/clause reduction is augmented to succeed if the guard also succeeds.

Remark 2.17 (Guards and SRSW). Guard occurrences count toward SRSW satisfaction: if $X?$ occurs in a guard, its paired writer X must occur in the head and $X?$ may additionally occur once in the body. Furthermore, if the success of a guard implies that $X?$ is bound to a ground term, then both X and $X?$ may occur multiple times in the clause. Groundness-implying guards include *ground*, *integer*, *number*, *string*, *constant*, arithmetic comparisons ($<$, $>$, $=<$, $>=$, $==$, $=\backslash=$), and ground equality ($=?=$). However, *known* and *compound* do not imply groundness.

Remark 2.18 (Anonymous Variables). An *anonymous variable* is any variable whose name begins with $_$ (e.g., $_$, $_Out$). Anonymous variables may appear anywhere a writer variable may appear; each occurrence denotes a fresh writer with no paired reader. Anonymous readers ($_?$, $_Name?$) are not permitted.

Guard predicates include type tests (*integer*, *number*, *string*, *constant*, *compound*, *ground*, *known*), arithmetic comparisons ($<$, $>$, $=<$, $>=$, $==$, $=\backslash=$), and ground equality ($=?=$).

System predicates and body kernels. GLP *system predicates* such as $:=$ (arithmetic assignment), *now* (clock access), and $=.$ (term composition/decomposition) are predicates defined by GLP clauses whose bodies may invoke *body kernel predicates*—runtime-implemented primitives not directly accessible to user programs. The system predicate’s own guards ensure that body kernel preconditions are met before invocation. For example, arithmetic assignment is defined recursively by clauses that pattern-match on each operator:

```
Result? := N :- number(N?) | Result = N?.
Result? := X + Y :- number(X?), number(Y?) |
    '_add'(X?, Y?, Result).
Result? := X + Y :- otherwise |
    X1 := X?, Y1 := Y?, Result := X1? + Y1?.
```

The base case binds a plain number; the first addition clause guards that both operands are numbers and invokes the `'_add'` body kernel; the fallback clause recursively evaluates subexpressions. The complete definitions of all system predicates and the full table of body kernels appear in the extended (arXiv) version of this paper.

2.4 GLP Programming Techniques

On the one hand, GLP has no unification, only pattern-matching and single-writer variables. On the other hand, goal reduction may write on several variables atomically. The result is that almost the entire range of concurrent logic programming techniques developed since the 1980s [29, 28] is available in

GLP, including: streams and their fair, biased, and self-balancing merge [41, 42]; channels as generalised streams [48]; messages with reply variables and bounded buffers [46]; network reconfiguration and distributed programming [25]; object-oriented programming with mutable state via stream-recursion [43]; systems programming and computation control [27, 45]; and the full range of meta-interpreters — fail-safe, termination-detecting, tracing, and algorithmic debugging [24, 21].

Two techniques require workarounds. The use of logic variables for mutual exclusion — standard in concurrent logic languages with atomic unification [29] — must be achieved in GLP via a monitor [13, 2], the standard distributed-computing solution. Similarly, general broadcast — distributing a value to multiple consumers — is not directly supported by GLP, since it would require multiple occurrences of the same reader, in violation of SO; it must be achieved by explicit *distributors*. We note that if moded-type checking can determine that the values to be distributed never contain writers, then the SO restriction on readers may safely be relaxed allowing broadcast natively; a possible future upgrade of the language. Excluded is the encoding of logic programs or-parallel by the and-parallelism of Concurrent Prolog [7, 30], which depends on general unification in a fundamental way.

3 Multiagent GLP

This section extends GLP to multiple agents. We first recall the notion of multiagent transition systems via multiagent atomic transactions [31, 38], then define multiagent GLP (maGLP) as a transactions-based multiagent transition system, and finally show how the social graph operates in the multiagent setting.

3.1 Multiagent Transition Systems

We assume a potentially infinite set of *agents* Π , but consider only finite subsets of it, so when we refer to a particular set of agents $P \subset \Pi$ we assume P to be nonempty and finite. We use $\subset$ to denote the strict subset relation and $\subseteq$ when equality is also possible.

We use S^P to denote the set S indexed by the set P , and if $c \in S^P$ we use c_p to denote the member of c indexed by $p \in P$. Intuitively, think of such a $c \in S^P$ as an array of cells indexed by members of P with cell values in S .

Definition 3.1 (Local States, Configuration, Transaction, Participants). Given agents $Q \subset \Pi$ and an arbitrary set S of *local states*, a *configuration* over Q and S is a member of $C := S^Q$. An *atomic transaction*, or just *transaction*, over Q and S is any pair of configurations $t = c \rightarrow c' \in C^2$ such that $c \neq c'$, with $t_p := c_p \rightarrow c'_p$ for any $p \in Q$, and with p being an *active participant* in t if $c_p \neq c'_p$, *stationary participant* otherwise.

Definition 3.2 (Degree). The *degree* of a transaction t (unary, binary, $\dots$, k -ary) is the number of active participants in t , and the *degree* of a set of transactions T is the maximal degree of any $t \in T$.

Definition 3.3 (Multiagent Transition System). Given agents $P \subset \Pi$ and an arbitrary set S of *local states* with a designated *initial local state* $s_0 \in S$, a *multiagent transition system* over P and S is a transition system $TS = (C, c_0, T)$ with *configurations* $C := S^P$, *initial configuration* $c_0 := \{s_0\}^P$, and *transitions* $T \subseteq C^2$ being a set of transactions over P and S , with the *degree* of TS being the degree of T .

Rather than specifying a multiagent transition system over a set of agents P directly, we specify it via atomic transactions, which are typically of bounded degree smaller than $|P|$.

Definition 3.4 (Transaction Closure). Let $P \subset \Pi$, S a set of local states, and $C := S^P$. For a transaction $t = (c \rightarrow c')$ over local states S with participants $Q \subseteq P$, the P -closure of t , $t \uparrow P$, is the set of transitions over P and S defined by:

$$t \uparrow P := \{t' \in C^2 : \forall q \in Q. (t_q = t'_q) \wedge \forall p \in P \setminus Q. (p \text{ is stationary in } t')\}$$

If R is a set of transactions, each $t \in R$ over some $Q \subseteq P$ and S , then the P -closure of R , $R \uparrow P$, is the set of transitions over P and S defined by:

$$R \uparrow P := \bigcup_{t \in R} t \uparrow P$$

Namely, the closure over $P \supseteq Q$ of a transaction t over Q includes all transitions t' over P in which members of Q do the same in t and in t' , and the rest remain in their current (arbitrary) state.

Definition 3.5 (Transactions-Based Multiagent Transition System). Given agents $P \subset \Pi$, local states S with initial local state $s0 \in S$, and a set of transactions R , each $t \in R$ over some $Q \subseteq P$ and S , the *transactions-based multiagent transition system* over P , S , and R is the multiagent transition system $TS = (S^P, \{s0\}^P, R \uparrow P)$.

In other words, one can fully specify a multiagent transition system over S and P simply by providing a set of atomic transactions over S , each with participants $Q \subseteq P$.

3.2 From cGLP to maGLP

In extending GLP to multiple agents, each agent maintains its own asynchronous resolvent as its local state. The key insight is that GLP's variable pairs provide natural binary communication channels: when agent p assigns a writer X for which the paired reader $X?$ is held by agent q , the assignment $X := T$ must be communicated to q .

A key difference between cGLP and maGLP is in the initial state. In a multiagent transition system all agents must have the same initial local state $s0$ (Definition 3.3). This precludes setting up an initial configuration in which agents share logic variables, as this would imply different initial states for different agents.

We resolve this in two steps. First, we employ only anonymous logic variables “_” in the initial local states of agents: Anonymous variables are, on the one hand, syntactically identical, hence allow all initial states to be syntactically identical, and on the other hand represent unique variables, hence semantically all initial goals have unique, local, non-shared variables. The initial state of all agents is the atomic goal $\text{agent}(\text{ch}(_, _), \text{ch}(_, _))$, providing two bidirectional channels: the first to the person operating the machine, and the second to the network. The first channel is a bidirectional stream of GLP messages between the machine and the person, mediated by runtime UI support below the GLP layer and by physical UI hardware; messages flowing toward the person are rendered as UI elements, and the person's responses flow back as GLP messages. With appropriate UI runtime support, a GLP message to the user may contain a writer, in which case the runtime presents it as a question and an eventual response from the person becomes the corresponding assignment. The second channel carries communication with other agents.

Second, the Cold-call transaction enables agents to bootstrap communication by establishing shared variables through the network infrastructure, realizing the cold-call protocol for connecting previously-disconnected agents.

3.3 Multiagent GLP Definition

Definition 3.6 (Multiagent GLP). Given agents $P \subset \Pi$ and GLP program M , the *maGLP transition system* over P and M is the transactions-based multiagent transition system (Definition 3.5) over P , local states being asynchronous resolvents (G_p, σ_p) over M , initial local state $s_0 = (\{\text{agent}(\text{ch}(_, _), \text{ch}(_, _))\}, \emptyset)$, and the following transactions $c \rightarrow c'$:

1. **Reduce p :** A unary transaction with participant p where $c_p \rightarrow c'_p$ is a cGLP Reduce transition (Definition 2.10).
2. **Communicate p to q :** A transaction with participants $p, q \in P$ where $\{X? := T\} \in \sigma_p$, $X?$ occurs in G_q , $\sigma'_p = \sigma_p \setminus \{X? := T\}$, and $c'_q = (G_q\{X? := T\}, \sigma_q)$.
3. **Cold-call p to q :** A binary transaction with participants $p \neq q \in P$ where the network output stream in c_p has a new message $\text{msg}(q, X)$, c'_p is the result of advancing the network output stream in c_p , and c'_q is the result of adding X to the network input stream in c_q .

Note that Communicate may be unary or binary, depending on whether $p = q$. Communicate transfers assignments from writers to readers between agents sharing a paired reader and writer. Cold-call transfers a term with its variables to q through the network streams established in each agent's initial configuration, enabling the creation of paired variables among previously-disconnected agents. Naturally, Cold-call would be the exception, as once agents share a paired variable they can use it to communicate indefinitely; moreover, an agent with two friends (with which it shares channels) may introduce them to each other, also eschewing the need for a Cold-call.

Transaction Equivalence for maGLP. Following the cGLP transition equivalence (Definition 2.14), we extend the equivalence relation to maGLP transactions. Two maGLP transactions $t_1 \sim t_2$ if:

- both are Reduce transactions at the same agent p , reducing the same goal (by identity) with the same clause, or
- both are Communicate transactions from the same p to the same q , applying the counterpart of the same writer assignment to the same goal (by identity) at q , or
- both are Cold-call transactions from the same p to the same q , delivering the same message from p 's network output stream.

3.4 The Grassroots Social Graph

The grassroots social graph is the foundation upon which all other grassroots platforms are built. Nodes represent cryptographically self-identified agents; edges represent authenticated bidirectional channels; connected components arise spontaneously through befriending.

Each agent processes messages from user and network input streams and maintains an outputs list mapping named destinations to output streams. The program supports three protocols: (1) *cold-call befriending*, where agents with no prior shared variables establish friendship by exchanging a response variable through the network; (2) *text messaging* between established friends via named output streams; and (3) *friend-mediated introduction*, where a mutual friend creates a channel pair and sends each half to the respective parties, establishing a direct connection.

The core of the program is the procedure `agent/4`; three of its clauses follow.

Channels. A *channel* is an authenticated bidirectional communication structure `ch(In, Out)`, where `In` is a stream of incoming messages and `Out` is a stream of outgoing messages. Two agents share a channel by holding opposite views of the same underlying stream pair: what one writes on its `Out`, the other reads from its `In`. The procedure `new_channel/2` creates such a pair:

```
new_channel(ch(Xs?, Ys), ch(Ys?, Xs)).
```

A single call `new_channel(PQCh, QPCh)` binds `PQCh` to `ch(Xs?, Ys)` and `QPCh` to `ch(Ys?, Xs)`: the reader `Xs?` of one channel is paired with the writer `Xs` of the other (and likewise for `Ys/Ys?`), so each end's outgoing stream is the other end's incoming stream. Sending and receiving on a channel are accordingly:

```
send(X, ch(In, [X?|Out?]), ch(In?, Out)).
receive(X?, ch([X|In], Out?), ch(In?, Out)).
```

Channels are used directly in friend-mediated introduction (below) to establish a direct connection between two parties without a cold-call. They are also used internally by the cold-call protocol: when the recipient accepts, it returns a freshly-created channel half as its response, completing the bidirectional link.

Cold-call befriending. The user's `connect(Target)` request triggers a cold-call: the agent sends `intro(Id, Resp)` on the network and injects the paired reader `Resp?` into its own user-input stream, so the eventual response surfaces locally as a normal user-side message:

```
agent(Id, [msg('_user', Id1, connect(Target))|UserIn], NetIn, Outs) :-
    Id? == Id1?, ground(Target?) |
    lookup_send('_net', msg(Target?, intro(Id?, Resp)), Outs?, Outs1),
    inject_msg(Resp?, Target?, Id?, UserIn?, UserIn1),
    agent(Id?, UserIn1?, NetIn?, Outs1?).
```

On the recipient side, an incoming `intro(From, Resp)` is forwarded to the user as a befriend prompt; the user's decision will assign `Resp`, transparently transported back to the originator:

```
agent(Id, UserIn, [msg(Id1, intro(From, Resp))|NetIn], Outs) :-
    Id? == Id1? |
    lookup_send('_user', msg(agent, '_user', befriend(From?, Resp?)), Outs?, Outs1),
    agent(Id?, UserIn?, NetIn?, Outs1?).
```

When Alice executes this protocol to befriend Bob, her agent sends `msg(bob, intro(alice, Resp))` on the `'_net'` output stream. The Cold-call transaction (Definition 3.6) transfers this message to Bob's network input stream, adding the writer `Resp` to Bob's resolvent. When Bob accepts, assigning `Resp` to `accept(ch(FIn?, FOut))`, the Communicate transaction transfers this assignment back to Alice.

Friend-mediated introduction. Friend-mediated introduction creates a fresh channel pair via `new_channel/2` and sends one half to each of the introduced parties — a direct connection arises without a cold-call:

```
agent(Id, [msg('_user', Id1, introduce(P, Q))|UserIn], NetIn, Outs) :-
    Id? == Id1?, ground(P?), ground(Q?), new_channel(PQCh, QPCh) |
    lookup_send(P?, msg(Id?, P?, intro(Q?, QPCh?)), Outs?, Outs1),
    lookup_send(Q?, msg(Id?, Q?, intro(P?, PQCh?)), Outs1?, Outs2),
    agent(Id?, UserIn?, NetIn?, Outs2?).
```

When Bob introduces Alice to Charlie, the channel pair he creates contains readers that are transferred to Alice and Charlie via Communicate, establishing their direct connection without any Cold-call.

Boot and deployment. Each agent runs on a separate isolate (later — separate smartphone). Two boot variants — a headless boot with actors and an interactive UI boot — appear in the extended (arXiv) version of this paper, along with the complete program (helper predicates, actor implementations) and a comprehensive Alice/Bob/Charlie scenario in the extended (arXiv) version of this paper.

3.5 Safety Properties of maGLP

The safety properties established for cGLP in Section 2 extend to maGLP. SO preservation (cf. Proposition 2.12) generalizes directly:

Proposition 3.7 (maGLP SO Preservation). *If the initial goals of all agents satisfy SO, then every goal in every agent’s resolvent throughout the run satisfies SO.*

To relate maGLP to deduction, consider the *lifted system L*: the cGLP transition system whose resolvent is the union of all agents’ local resolvents, whose initial goal includes a `network` goal with channels paired to each agent’s network channels, and whose program is M augmented with the GLP definition of `network`.

Proposition 3.8. *maGLP runs correspond to cGLP runs of L , and their outcomes are logical consequences of the augmented program.*

Definition 3.9 (maGLP Fair Run). A complete maGLP run is *fair* if for every equivalence class $[t] \in T/\sim$: whenever $[t]$ becomes enabled, eventually some $t' \in [t]$ is taken.

Implementation. The implementation-ready deterministic variants of GLP and maGLP (the latter deterministic at the agent, not system, level), along with formal correctness proofs that they implement their respective specifications, are presented in a companion paper [39].

4 Multiagent GLP is Grassroots

This section establishes that maGLP is grassroots. We define the notion of grassroots following [20]; formal definitions appear in the extended (arXiv) version of this paper.

Informally, a protocol is *grassroots* if two disjoint groups of agents can each operate independently—their interleaved correct runs are correct runs of the combined system—yet the combined system offers genuinely new behaviours that neither group could produce on its own. The first requirement is *obliviousness*; the second is *interactivity*.

The Cold-call transaction (Definition 3.6) is the fundamental interactive transaction of maGLP. It allows an agent q in one group to connect to an agent p in another, establishing shared variables that span both groups—a behaviour that no interleaving of independent runs of the two groups could produce.

Theorem 4.1. *The maGLP protocol is grassroots.*

The grassroots property of maGLP extends to applications built on top of it, provided they use the cold-call mechanism.

Definition 4.2 (GLP Application). A *GLP application* is a GLP program M together with the maGLP infrastructure. An application *uses cold calls* if agents can execute the Cold-call transaction to establish communication with previously-disconnected agents.

Proposition 4.3. *Any GLP application that uses cold calls is grassroots.*

Corollary 4.4. *The GLP implementation of the grassroots social graph (Section ??) is grassroots.*

5 Related Work

Concurrent Logic Programming. GLP belongs to the family of concurrent logic programming (CLP) languages that emerged in the 1980s: Concurrent Prolog [26], GHC [49], and PARLOG [6]. These languages interpret goals as concurrent processes communicating through shared logical variables, using committed-choice execution with guarded clauses. Shapiro’s comprehensive survey [29] documents this family and its design space. A key evolution was *flattening*: restricting guards to primitive tests only. Flat Concurrent Prolog (FCP) [23] and Flat GHC [49] demonstrated that flat guards suffice for practical parallel programming while dramatically simplifying semantics and implementation.

GLP can be understood as FCP with the Single-Occurrence (SO) invariant. Concurrent Prolog introduced read-only annotations (?) distinguishing readers from writers of shared variables, enabling dataflow synchronization. However, read-only unification proved semantically problematic: Levi and Palamidessi [17] showed it is order-dependent, and Mierowsky et al. [23] documented non-modularity issues. GHC dispensed with read-only annotations entirely, relying on guard suspension semantics. GLP’s SO invariant—requiring that each variable occurs at most once in a goal or clause—resolves these difficulties by ensuring that (1) no races occur on variable assignment, and (2) term matching suffices, eschewing unification entirely. The result is a cleaner semantic foundation while preserving the expressiveness of stream-based concurrent programming. GLP retains logic programming’s metaprogramming capabilities [24, 21, 27], essential for platform tooling development.

Modes in Concurrent Logic Programming. Mode systems for CLP have a rich history. PARLOG used mode declarations at the predicate level, with input modes enforcing one-way matching. Ueda’s work on moded Flat GHC [50, 52] is most directly relevant: his mode system assigns polarity to every variable occurrence (positive for input/read, negative for output/write), with the *well-modedness* property guaranteeing each variable is written exactly once. Ueda’s subsequent linearity analysis [51] identifies variables read exactly once, enabling compile-time garbage collection. GLP enforces single-occurrence universally as a syntactic restriction, whereas Ueda’s system guarantees single-writer with single-reader as an optional refinement.

Linear logic and futures/promises. GLP’s SO invariant has deep connections to two foundational ideas. Girard’s linear logic [11] introduced the principle that resources must be used exactly once; GLP’s SO invariant enforces this at the variable level, ensuring each assignment is produced exactly once and consumed exactly once. Futures and promises [1, 9] introduced placeholders for values computed asynchronously; GLP’s reader/writer pairs realize this mechanism, with the writer as promise (single producer) and reader as future (single consumer). The line of work by Caires et al. [3, 4] established deep connections between linear logic and session types; GLP achieves similar resource-discipline properties through syntactic restrictions rather than type systems.

Grassroots platforms. The mathematical foundations for grassroots platforms were established in [32], with atomic transactions introduced in [38]. Grassroots social networks [33], cryptocurrencies [34, 18], and federations [12, 47] have been formally specified and proven grassroots. GLP provides the programming language to implement these specifications.

Companion papers. This paper is the founding paper on GLP. Based on it, a companion paper [40] developed moded types for GLP, extending LP types [10] with modes that capture directionality of communication, enabling typing of interactive partial computations. Another companion paper [39] derives a deterministic implementation madGLP from maGLP and proves madGLP grassroots.

6 Conclusion

While concurrent logic programming and its powerful programming techniques have been known for four decades [29], adoption has been hampered by their inaccessibility to the average programmer; with AI as a hyper-programmer this limitation is removed. Our experience to date is that AI is highly effective and productive at programming from mathematical specifications, particularly when the target is a typed high-level language such as GLP, where types constrain the space of legal programs and provide a checkable specification at the human-AI interface [40]. Moreover, our experience suggests that the abstract nature of concurrent logic programming renders it a more powerful and productive language than mainstream languages for collaborative human-AI abstract specifications-based program development.

Constitutional governance of grassroots communities and federations builds on Constitutional Consensus [14, 19] and digital social contracts [5, 44], both realisable in GLP. A textbook on GLP and programming in it is in preparation. The GLP runtime and example programs are open-source at <https://github.com/EShapiro2/GLP>.

References

- [1] Henry G. Baker & Carl Hewitt (1977): *The Incremental Garbage Collection of Processes*. In: *Proceedings of the 1977 Symposium on Artificial Intelligence and Programming Languages*, ACM, pp. 55–59, doi:10.1145/800228.806932.
- [2] Per Brinch Hansen (1973): *Operating System Principles*. Prentice-Hall.
- [3] Luís Caires & Frank Pfenning (2010): *Session Types as Intuitionistic Linear Propositions*. In: *Proceedings of the 21st International Conference on Concurrency Theory (CONCUR)*, Lecture Notes in Computer Science 6269, Springer, pp. 222–236, doi:10.1007/978-3-642-15375-4_16.
- [4] Luís Caires, Frank Pfenning & Bernardo Toninho (2016): *Linear Logic Propositions as Session Types*. *Mathematical Structures in Computer Science* 26(3), pp. 367–423, doi:10.1017/S0960129514000218.
- [5] Luca Cardelli, Liav Orgad, Gal Shahaf, Ehud Shapiro & Nimrod Talmon (2020): *Digital social contracts: A foundation for an egalitarian and just digital society*. In: *CEUR Proceedings of the First International Forum on Digital and Democracy*, 2781, CEUR-WS, pp. 51–60.
- [6] Keith Clark & Steve Gregory (1986): *PARLOG: parallel programming in logic*. *ACM Transactions on Programming Languages and Systems (TOPLAS)* 8(1), pp. 1–49.
- [7] Michael Codish & Ehud Shapiro (1986): *Compiling OR-parallelism into AND-parallelism*. In: *International Conference on Logic Programming*, Springer, pp. 283–297.
- [8] Ohad Eitan, Idit Keidar & Ehud Shapiro (2026): *Securing People and their Personal Machines Against Major Faults*. Submitted, arXiv XXXX.XXXXX.
- [9] Daniel P. Friedman & David S. Wise (1976): *The Impact of Applicative Programming on Multiprocessing*. *Indiana University Computer Science Department Technical Report (TR-26)*.
- [10] Thom Frühwirth, Ehud Shapiro, Moshe Y. Vardi & Eyal Yardeni (1991): *Logic Programs as Types for Logic Programs*. In: *Proceedings of the 6th Annual IEEE Symposium on Logic in Computer Science (LICS)*, IEEE Computer Society, pp. 300–309, doi:10.1109/LICS.1991.151654.
- [11] Jean-Yves Girard (1987): *Linear Logic*. *Theoretical Computer Science* 50(1), pp. 1–101, doi:10.1016/0304-3975(87)90045-4.
- [12] Daniel Halpern, Ariel D Procaccia, Ehud Shapiro & Nimrod Talmon (2024): *Federated Assemblies*. *Proc AAAI 2025*; arXiv preprint arXiv:2405.19129.
- [13] C. A. R. Hoare (1974): *Monitors: An operating system structuring concept*. *Communications of the ACM* 17(10), pp. 549–557.

- [14] Idit Keidar, Andrew Lewis-Pye & Ehud Shapiro (2025): *Constitutional Consensus*. arXiv preprint arXiv:2505.19216.
- [15] Anne-Marie Kermarrec, Erick Lavoie & Christian Tschudin (2020): *Gossiping with append-only logs in secure-Scuttlebutt*. In: *Proceedings of the 1st international workshop on distributed infrastructure for common good*, pp. 19–24.
- [16] Robert Kowalski (1974): *Predicate logic as programming language*. In: *IFIP congress*, 74, pp. 569–574.
- [17] Giorgio Levi & Catuscia Palamidessi (1985): *The Semantics of the Read-Only Variable*. In: *Proc. Symposium on Logic Programming*, IEEE, pp. 128–137.
- [18] Andrew Lewis-Pye, Oded Naor & Ehud Shapiro (2023): *Grassroots Flash: A Payment System for Grassroots Cryptocurrencies*. arXiv preprint arXiv:2309.13191.
- [19] Andrew Lewis-Pye & Ehud Shapiro (2025): *Morpheus Consensus: Excelling on trails and autobahns*. *Proc. 29th Conference on Principles of Distributed Systems, OPODIS 2025*. arXiv preprint arXiv:2502.08465, doi:10.4230/LIPIcs.OPODIS.2025.35.
- [20] Andy Lewis-Pye & Ehud Shapiro (2026): *Volitional Multiagent Atomic Transactions: Describing People and their Machines*. arXiv preprint arXiv:2604.25596.
- [21] Yossi Lichtenstein & Ehud Shapiro (1988): *Concurrent algorithmic debugging*. *ACM SIGPLAN Notices* 24(1), pp. 248–260.
- [22] John W. Lloyd (1987): *Foundations of Logic Programming*, 2nd edition. Springer-Verlag, doi:10.1007/978-3-642-83189-8.
- [23] C. Mierowsky, S. Taylor, E. Shapiro, J. Levy & M. Safra (1985): *On the implementation of Flat Concurrent Prolog*. *Proceedings of the 1985 Symposium on Logic Programming*, pp. 276–286.
- [24] Shmuel Safra & Ehud Shapiro (1988): *Meta interpreters for real*. In: *Concurrent Prolog: Collected Papers*, MIT Press, pp. 166–179.
- [25] A. Shafrir & E. Shapiro (1987): *Distributed Programming in Concurrent Prolog*. In Ehud Shapiro, editor: *Concurrent Prolog: Collected Papers, Volume 1*, MIT Press, Cambridge, MA, pp. 318–338.
- [26] Ehud Shapiro (1983): *A subset of Concurrent Prolog and its interpreter*. ICOT Technical Report, TR-003.
- [27] Ehud Shapiro (1984): *Systems programming in concurrent prolog*. In: *Proceedings of the 11th ACM SIGACT-SIGPLAN symposium on Principles of Programming Languages*, pp. 93–105.
- [28] Ehud Shapiro (1987): *Concurrent Prolog: collected papers (Vols. I and II)*. MIT press.
- [29] Ehud Shapiro (1989): *The family of concurrent logic programming languages*. *ACM Computing Surveys (CSUR)* 21(3), pp. 413–510.
- [30] Ehud Shapiro (1989): *Or-parallel prolog in flat concurrent prolog*. *The Journal of Logic Programming* 6(3), pp. 243–267.
- [31] Ehud Shapiro (2021): *Multiagent Transition Systems: Protocol-Stack Mathematics for Distributed Computing*. arXiv preprint arXiv:2112.13650.
- [32] Ehud Shapiro (2023): *Grassroots Distributed Systems: Concept, Examples, Implementation and Applications (Brief Announcement)*. In: *37th International Symposium on Distributed Computing (DISC 2023)*. (Extended version: arXiv:2301.04391), LIPICS, Italy, pp. 47:1, 47:7.
- [33] Ehud Shapiro (2023): *Grassroots Social Networking: Serverless, Permissionless Protocols for Twitter/LinkedIn/WhatsApp*. In: *OASIS '23*, Association for Computing Machinery, doi:10.1145/3599696.3612898.
- [34] Ehud Shapiro (2024): *Grassroots Currencies: Foundations for Grassroots Digital Economies*. arXiv preprint arXiv:2202.05619.
- [35] Ehud Shapiro (2025): *Characterising Global Platforms: Centralised, Decentralised, Federated, and Grassroots*. arXiv preprint arXiv:2511.03286.
- [36] Ehud Shapiro (2026): *Child-Safe Social Networking*. Submitted, arXiv XXXX.XXXXX.

- [37] Ehud Shapiro (2026): *Grassroots Bonds: A Grassroots Foundation for Market Liquidity*. arXiv preprint arXiv:2603.13671.
- [38] Ehud Shapiro (2026): *Grassroots Platforms with Atomic Transactions: Social Graphs, Cryptocurrencies, and Democratic Federations*. In: *Proceedings of the 27th International Conference on Distributed Computing and Networking*, pp. 71–81, doi:10.1145/3772290.3772309. ArXiv preprint arXiv:2502.11299.
- [39] Ehud Shapiro (2026): *Implementing Grassroots Logic Programs with Multiagent Transition Systems and AI*. arXiv preprint arXiv:2602.06934.
- [40] Ehud Shapiro (2026): *Types for Grassroots Logic Programs*. arXiv preprint arXiv:2601.17957.
- [41] Ehud Shapiro & Colin Mierowsky (1984): *Fair, biased, and self-balancing merge operators: Their specification and implementation in Concurrent Prolog*. *New Generation Computing* 2(3), pp. 221–240.
- [42] Ehud Shapiro & Shmuel Safra (1986): *Multiway merge with constant delay in Concurrent Prolog*. *New Generation Computing* 4(2), pp. 211–216.
- [43] Ehud Shapiro & Akikazu Takeuchi (1983): *Object oriented programming in Concurrent Prolog*. *New Generation Computing* 1(1), pp. 25–48.
- [44] Ehud Shapiro & Nimrod Talmon (2022): *Foundations for Grassroots Democratic Metaverse*. In: *Proceedings of the 21st International Conference on Autonomous Agents and Multiagent Systems, AAMAS '22*, International Foundation for Autonomous Agents and Multiagent Systems, Richland, SC, p. 1814–1818.
- [45] William Silverman, Michael Hirsch, Avshalom Houri & Ehud Shapiro (1988): *The Logix system user manual Version 1.21*. In: *Concurrent Prolog: Collected Papers*, pp. 46–77.
- [46] A. Takeuchi & K. Furukawa (1987): *Bounded Buffer Communication in Concurrent Prolog*. In Ehud Shapiro, editor: *Concurrent Prolog: Collected Papers, Volume 1*, MIT Press, Cambridge, MA, pp. 464–475.
- [47] Nimrod Talmon & Ehud Shapiro (2025): *Grassroots Federation: Fair Democratic Governance at Scale*. arXiv preprint arXiv:2505.02208; also *Proc. of AAMAS'26*.
- [48] E. D. Tribble, M. S. Miller, K. Kahn, D. G. Bobrow, C. Abbott & E. Shapiro (1987): *Channels: A Generalization of Streams*. In Ehud Shapiro, editor: *Concurrent Prolog: Collected Papers, Volume 1*, MIT Press, Cambridge, MA, pp. 446–463.
- [49] Kazunori Ueda (1986): *Guarded Horn Clauses*. In: *Logic Programming '85, Lecture Notes in Computer Science* 221, Springer, pp. 168–179.
- [50] Kazunori Ueda (1994): *Moded Flat GHC and Its Message-Oriented Implementation Technique*. *New Generation Computing* 12(4), pp. 337–368.
- [51] Kazunori Ueda (2001): *Resource-passing concurrent programming*. *Proceedings of TACS 2001*, pp. 95–126.
- [52] Kazunori Ueda & Masao Morita (1995): *I/O mode analysis in concurrent logic programming*. In: *Proceedings of the International Symposium on Theory and Practice of Parallel Programming*, Springer, pp. 356–368.